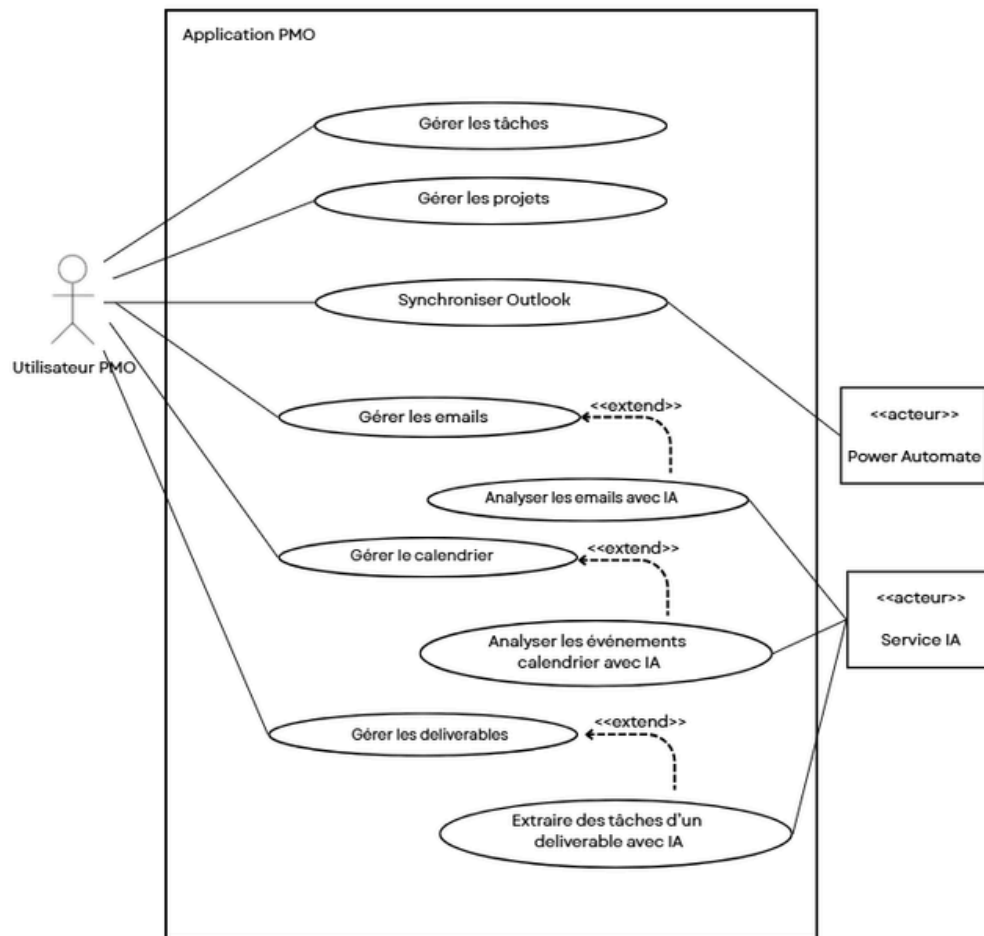


DIAGRAMMES



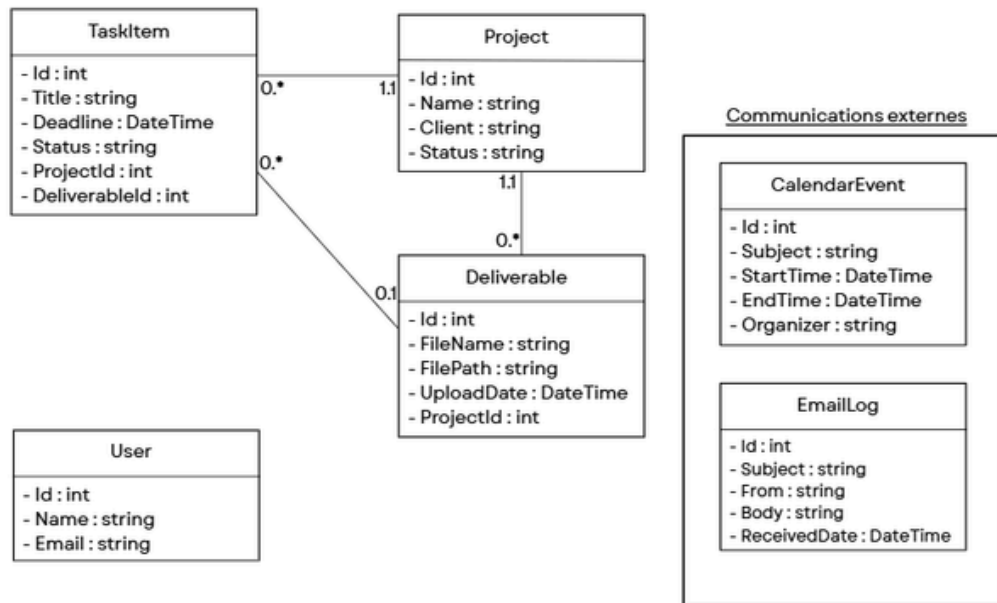
Use case

Le diagramme de use case présente une vue fonctionnelle globale de l'application PMO. Il permet de montrer les principaux acteurs qui interagissent avec le système ainsi que les fonctionnalités majeures offertes par l'application.

Dans ce diagramme, l'acteur principal est l'Utilisateur PMO, qui utilise l'application pour gérer les projets, les tâches, les deliverables, les emails et le calendrier. Le diagramme montre également l'intervention de deux acteurs externes : Power Automate, utilisé pour synchroniser les données Outlook, et le Service IA, utilisé pour enrichir certaines fonctionnalités par des analyses intelligentes.

Les cas d'utilisation représentés décrivent les grandes fonctions métier de l'application. L'utilisateur peut gérer les projets, gérer les tâches, gérer les deliverables, gérer les emails, gérer le calendrier et lancer la synchronisation Outlook. En complément, certaines fonctionnalités peuvent être enrichies par l'intelligence artificielle, comme l'analyse des emails, l'analyse des événements calendrier et l'extraction de tâches à partir d'un deliverable.

Ce diagramme met donc en évidence le rôle central de l'utilisateur dans l'application, l'intégration de services externes, ainsi que la manière dont l'IA et l'automatisation viennent compléter les fonctionnalités principales du système.



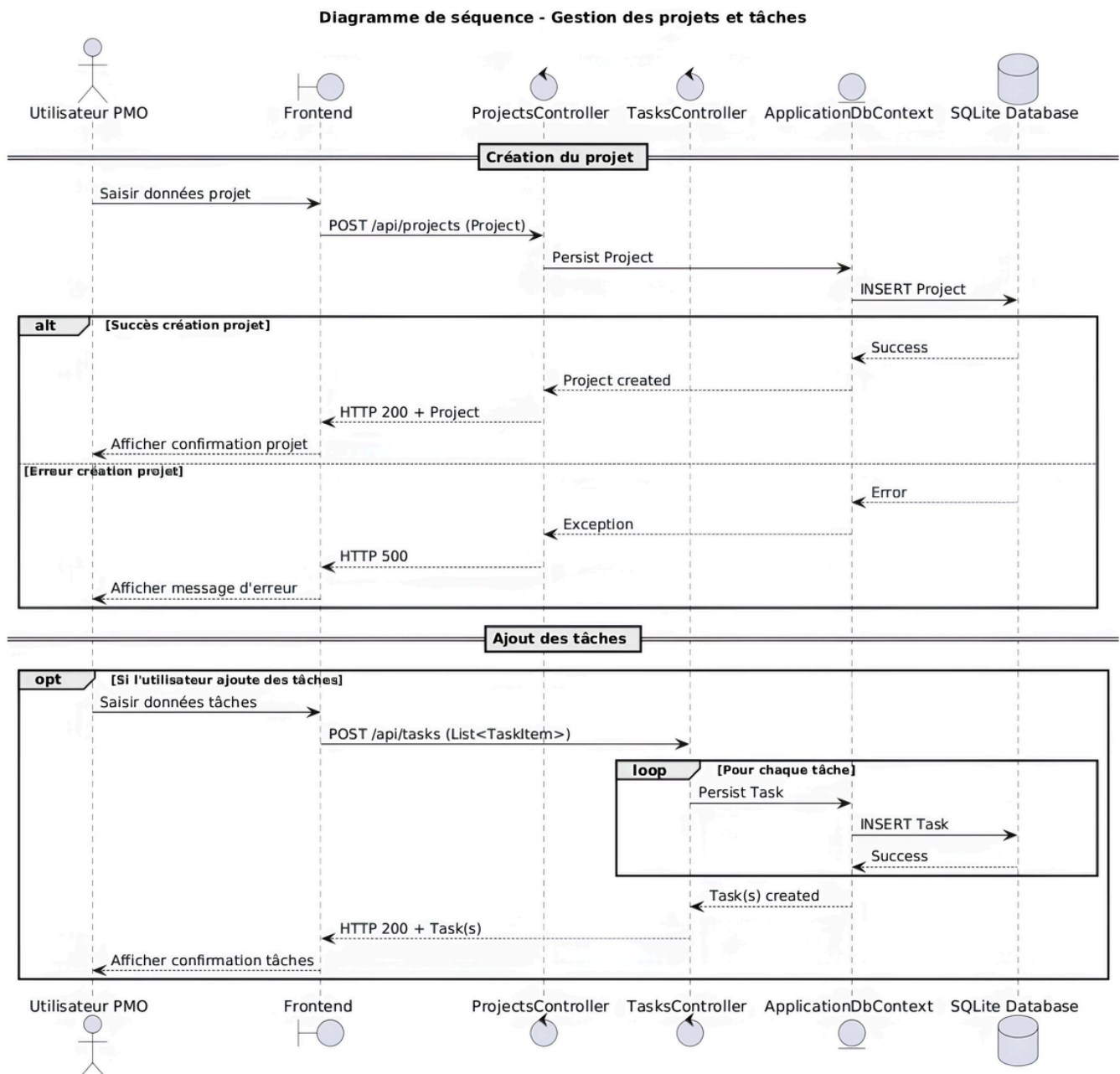
Classe

Le diagramme de classes présente la structure statique générale de l'application PMO. Il permet de représenter les principales entités métier du système, leurs attributs essentiels ainsi que les relations qui existent entre elles.

Dans ce diagramme, la classe centrale est Project, car elle structure l'organisation générale des informations métier. Un projet peut contenir plusieurs tâches (TaskItem) et plusieurs deliverables. La classe TaskItem représente les tâches à réaliser dans le cadre d'un projet, avec des informations comme le titre, la date limite et le statut. La classe Deliverable représente les documents ou livrables associés à un projet, avec des informations comme le nom du fichier, son chemin et sa date d'upload.

Le diagramme montre également d'autres classes importantes, comme User, qui représente l'utilisateur de l'application, ainsi que EmailLog et CalendarEvent, qui correspondent respectivement aux emails importés et aux événements calendrier synchronisés depuis l'extérieur. Ces deux classes sont représentées comme des entités indépendantes, car elles n'ont pas de relation métier directe forte avec les projets, les tâches ou les deliverables dans le modèle général.

Ce diagramme permet donc de comprendre comment les données principales de l'application sont organisées et reliées entre elles. Il donne une vision claire de la structure métier du système, en mettant en évidence les classes centrales et leurs associations principales.



sequence 1

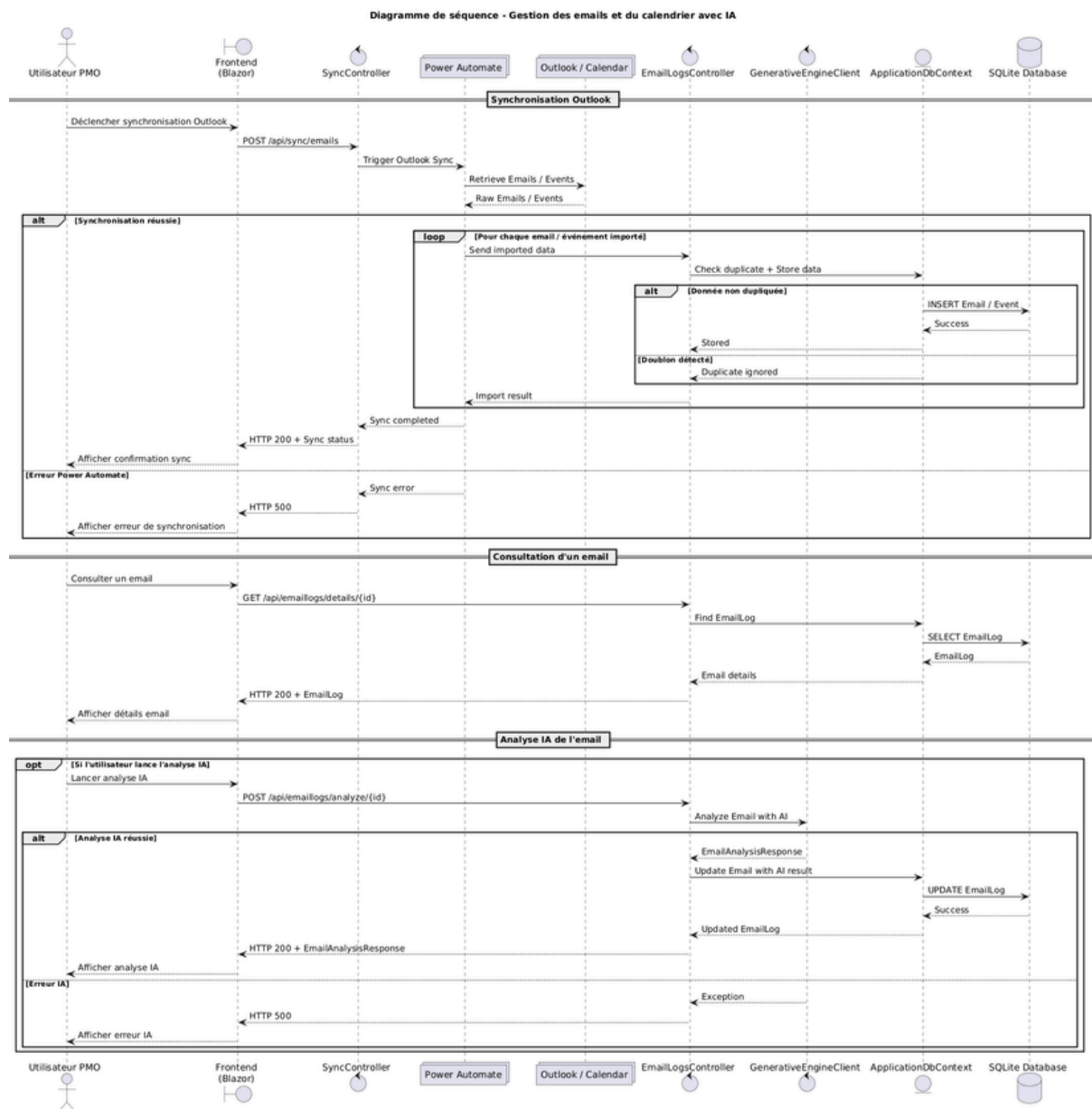
gestion des projets et taches

Le diagramme de séquence présente le déroulement dynamique de la fonctionnalité de gestion des projets et des tâches dans l'application PMO. Il permet de représenter, dans l'ordre chronologique, les différentes interactions entre l'utilisateur, l'interface frontend, les contrôleurs backend, le contexte de persistance et la base de données. Dans ce scénario, l'acteur principal est l'Utilisateur PMO, qui commence par saisir les informations d'un nouveau projet à travers l'interface de l'application. Le frontend transmet ensuite cette demande au ProjectsController, qui se charge de traiter la requête et de demander la persistance des données via ApplicationDbContext. Celui-ci communique avec la base de données SQLite afin d'enregistrer le projet. Une fois l'opération terminée, la réponse remonte vers le contrôleur puis vers le frontend, qui affiche une confirmation à l'utilisateur.

Le diagramme montre ensuite une deuxième phase, correspondant à l'ajout des tâches associées au projet créé. L'utilisateur saisit les informations des tâches dans l'interface, puis le frontend envoie ces données au TasksController. Comme plusieurs tâches peuvent être ajoutées, le diagramme met en évidence une répétition de traitement pour chaque tâche à enregistrer. Chaque tâche est ainsi transmise au DbContext, qui effectue l'insertion dans la base de données, avant de renvoyer le résultat au contrôleur puis au frontend.

Le diagramme fait également apparaître des cas alternatifs, notamment en cas d'erreur lors de la sauvegarde des données. Cette représentation permet de montrer que le système ne se limite pas à un fonctionnement linéaire, mais prend également en compte les cas d'échec possibles. De plus, l'ajout des tâches est modélisé comme une étape optionnelle, car un projet peut être créé même si aucune tâche n'est ajoutée immédiatement.

Ce diagramme de séquence permet donc de comprendre clairement la circulation des informations et la collaboration entre les différentes couches de l'application lors de la création d'un projet et de l'ajout de tâches. Il met en évidence la logique temporelle du traitement, depuis l'action de l'utilisateur jusqu'à l'enregistrement des données dans la base.



sequence 2

gestion des emails et calender

(avec l'ai)

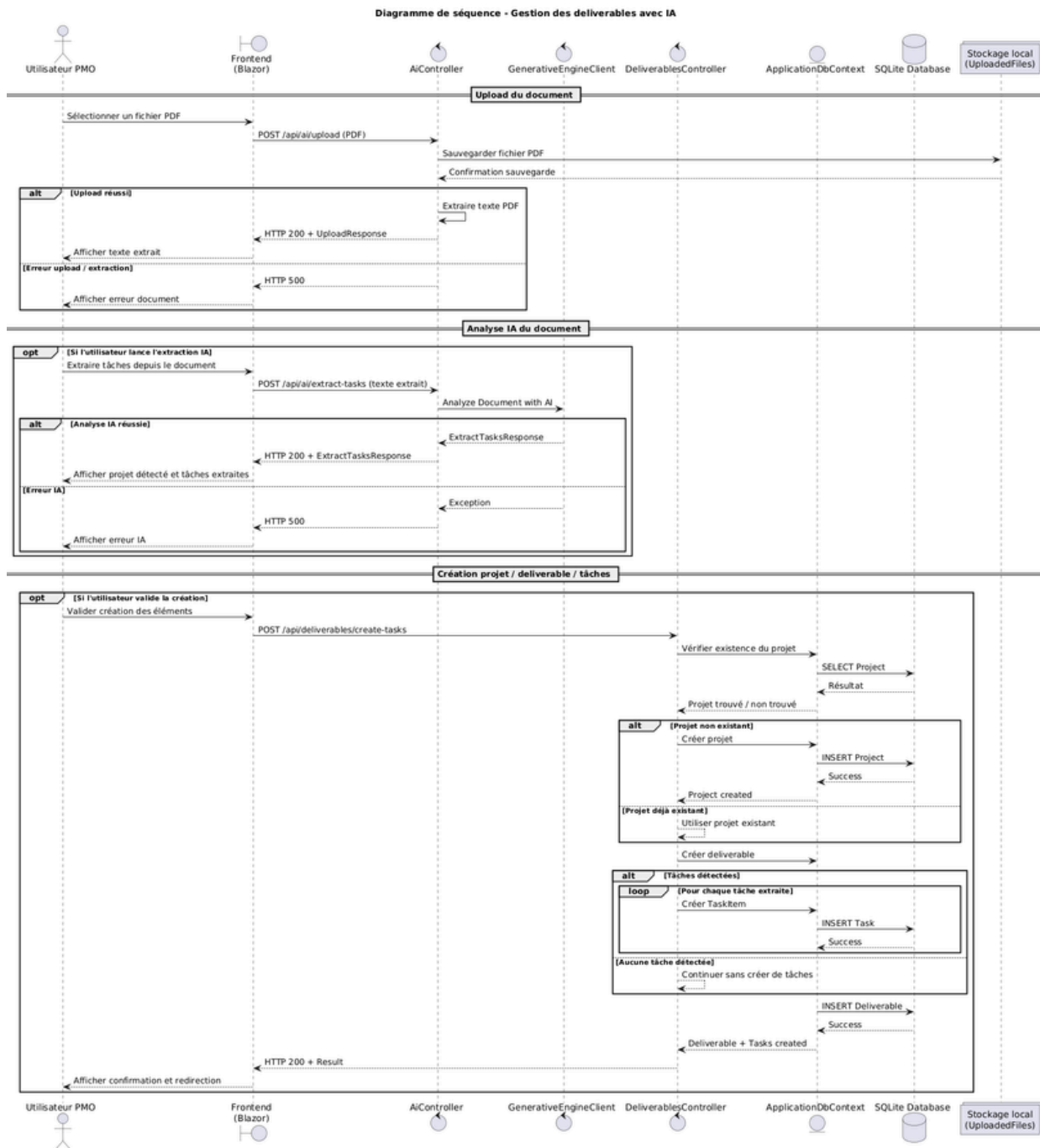
Le diagramme de séquence présente le déroulement dynamique de la gestion des emails et du calendrier avec intelligence artificielle dans l'application PMO. Il permet de représenter, dans l'ordre chronologique, les interactions entre l'utilisateur, l'interface frontend, les contrôleurs backend, le service d'automatisation Power Automate, le service IA, le contexte de persistance et la base de données.

Dans ce scénario, l'utilisateur PMO commence par déclencher la synchronisation Outlook à partir du frontend. Cette demande est transmise au SyncController, qui lance ensuite le flow Power Automate. Power Automate récupère les emails et les événements calendrier depuis Outlook / Calendar, puis les envoie à l'application pour stockage. Le diagramme montre que chaque donnée importée est traitée dans une boucle, car plusieurs emails ou événements peuvent être synchronisés. Pour chaque élément, l'application vérifie d'abord s'il existe déjà, afin d'éviter les doublons, puis l'enregistre dans la base de données si nécessaire. Une fois la synchronisation terminée, une confirmation est renvoyée jusqu'à l'utilisateur.

Dans une seconde phase, le diagramme montre la consultation d'un email importé. L'utilisateur sélectionne un email à partir du frontend, qui envoie alors une requête au EmailLogsController. Celui-ci interroge ApplicationDbContext, qui récupère les informations correspondantes depuis la base SQLite. Les détails de l'email sont ensuite renvoyés au frontend et affichés à l'utilisateur.

Le diagramme représente ensuite une étape optionnelle d'analyse par intelligence artificielle. Si l'utilisateur choisit de lancer l'analyse IA, le frontend envoie une requête au EmailLogsController, qui sollicite le service GenerativeEngineClient. Ce service effectue l'analyse de l'email, renvoie le résultat au contrôleur, puis les nouvelles informations sont enregistrées dans la base de données avant d'être affichées à l'utilisateur. Le diagramme prévoit également des cas alternatifs, notamment en cas d'erreur de synchronisation avec Power Automate, de détection de doublons, ou d'échec de l'analyse IA.

Ce diagramme de séquence permet ainsi de comprendre clairement comment l'application synchronise des données externes, les stocke, les déduplique, puis les enrichit grâce à l'intelligence artificielle. Il met en évidence la logique temporelle du traitement ainsi que la collaboration entre les différents composants du système.



sequence 3

gestion des deliverables (avec ai)

Le diagramme de séquence présente le déroulement dynamique de la gestion des deliverables avec intelligence artificielle dans l'application PMO. Il permet de représenter, dans l'ordre chronologique, les différentes interactions entre l'utilisateur, l'interface frontend, les contrôleurs backend, le service d'intelligence artificielle, le stockage local des fichiers, le contexte de persistance et la base de données.

Dans ce scénario, l'utilisateur PMO commence par sélectionner un document PDF depuis l'interface de l'application. Le frontend envoie alors ce fichier au AiController, qui se charge de le sauvegarder localement dans le dossier de stockage prévu. Une fois le fichier enregistré, le contrôleur procède à l'extraction du texte contenu dans le document. Le texte extrait est ensuite renvoyé au frontend afin d'être affiché à l'utilisateur.

Dans une seconde phase, le diagramme montre une étape optionnelle d'analyse par intelligence artificielle. Si l'utilisateur choisit de lancer l'extraction automatique des tâches, le frontend envoie le texte extrait au AiController, qui sollicite le service GenerativeEngineClient. Ce service effectue l'analyse du contenu du document et renvoie un résultat structuré contenant notamment le projet détecté et les tâches proposées. Le frontend affiche ensuite ces informations à l'utilisateur afin qu'il puisse les valider.

Le diagramme représente enfin la phase de création des éléments métier dans le système. Si l'utilisateur confirme, le frontend envoie les données au DeliverablesController. Celui-ci commence par vérifier si le projet existe déjà dans la base de données. Si ce n'est pas le cas, le projet est créé. Ensuite, le deliverable est enregistré, ainsi que les tâches extraites si elles existent. Le diagramme montre également qu'il peut y avoir plusieurs tâches à créer, ce qui est représenté par une boucle. Une fois les opérations terminées, une confirmation est renvoyée au frontend, qui informe l'utilisateur et peut le rediriger vers la page appropriée.

Ce diagramme de séquence permet ainsi de comprendre clairement comment l'application prend en charge l'automatisation de la gestion des deliverables, depuis l'upload d'un document jusqu'à la création des éléments métier liés. Il met en évidence la collaboration entre l'utilisateur, le frontend, les services d'analyse et la base de données, ainsi que les cas alternatifs possibles comme les erreurs d'upload, les erreurs d'analyse IA ou l'existence préalable du projet.